

Spring Boot Authentication & OTP Architecture

Complete Technical Guide: Registration, OTP Email Verification, JWT Login & Password Reset

1. Executive Overview

This service implements a production-grade, stateless security system using **Spring Security 6**, **JWT (JSON Web Tokens)**, **PostgreSQL persistence**, and **JavaMailSender (Gmail SMTP)**. User accounts require two-factor verification via secure 6-digit one-time passwords (OTP) delivered to the user's registered email before access is granted.

Component	Implementation Detail
Password Security	BCrypt strong hashing algorithm via BCryptPasswordEncoder
Session Management	Stateless JWT via JwtAuthFilter (Authorization: Bearer <token>)
OTP Generation	6-digit cryptographically secure number (SecureRandom), 5-minute expiry
Mail Transport	Gmail SMTP (smtp.gmail.com:587) with STARTTLS encryption
Database Engine	PostgreSQL dialect with JPA Hibernate auto-update DDL

2. User Registration & Email Verification Process

Account onboarding enforces email validation before allowing any login activity:

- **Step 1 (Submission):** The user submits POST `/api/auth/register` with `firstName`, `lastName`, `email`, and `password`.
- **Step 2 (Account Creation):** The system verifies the email is not already taken, hashes the password using BCrypt, and inserts the user record with `isVerified = false`.
- **Step 3 (OTP Dispatch):** A 6-digit random code is generated, stored in the `password_reset_otp` table with a 5-minute expiration timestamp, and emailed via Gmail SMTP.
- **Step 4 (OTP Verification):** The user submits POST `/api/auth/verify-registration-otp` with their email and 6-digit code.
- **Step 5 (Activation & Token):** The server verifies that the OTP is correct, unused, and unexpired. Upon validation, `isVerified` is updated to true, the OTP is marked used, and a signed **JWT AuthResponse** is returned.

3. Login & JWT Session Management

- **Step 1:** User submits POST `/api/auth/login` with email and password.
- **Step 2 (Verification Gate):** The service checks `user.isVerified()`. If false, it aborts immediately with 400 Bad Request: 'Please verify your email before logging in'.
- **Step 3 (Credential Match):** `AuthenticationManager` verifies the raw password against the stored BCrypt hash.
- **Step 4 (Token Generation):** `JwtUtil` generates a compact, cryptographically signed JWT token embedding username and USER authorities.

• **Step 5 (Protected Requests):** The client includes Authorization: Bearer <token> in all protected requests. JwtAuthFilter validates the signature on each request.

4. Forgot Password & Recovery Flow

- **Step 1 (Request Code):** User sends POST /api/auth/forgot-password with their registered email.
- **Step 2 (OTP Issuance):** An unguessable 6-digit OTP is generated, persisted with a 5-minute expiry, and emailed to the user.
- **Step 3 (Reset Password):** User sends POST /api/auth/reset-password with email, otpCode, and newPassword.
- **Step 4 (Execution):** The OTP is verified and marked used; the user's password is encrypted and updated in PostgreSQL.

5. API Endpoints Reference

Endpoint	Method	Request Payload	Success Response
/api/auth/register	POST	{firstName, lastName, email, password}	200 OK ('Registered. OTP sent...')
/api/auth/verify-registration-otp	POST	{email, otpCode}	200 OK (JWT AuthResponse)
/api/auth/login	POST	{email, password}	200 OK (JWT AuthResponse)
/api/auth/forgot-password	POST	{email}	200 OK ('OTP sent to email')
/api/auth/reset-password	POST	{email, otpCode, newPassword}	200 OK ('Password reset successfully')

■■ Security Safeguards:

- **Single-Use OTPs:** Once validated, OTPs are immediately flagged as used = true in the DB to prevent replay attacks.
- **Time-Bound Expiration:** Codes expire after exactly 5 minutes (LocalDateTime.now().plusMinutes(5)).
- **Salted BCrypt:** Passwords are never stored in plaintext and cannot be reverse-engineered.
- **Stateless Sessions:** No session memory is retained on the server, ensuring infinite horizontal scalability.